

Introduction

This specification is about a communication framework between tasks, based on message queues, established via brokers.

Tasks

Tasks follow a traditional event-oriented execution:

```
class Task {
    public abstract void post(Runnable r);
    public abstract void post(Runnable r, int delay);
    ...
}
```

with a traditional lifecycle:

```
class Task {
    ...
    public abstract boolean dead();
    public abstract void exit(Object o);
    public abstract void fail(Throwable th);

    public interface Listener {
        public void failed(Task t, Throwable th);
        public void completed(Task t, Object o);
    }
    public abstract void set(Listener l);
}
```

A dead task is a task that has either failed or exited with a given object or null. A task can be failed explicitly, invoking the method `fail` with an exception, but it may also fail implicitly if any posted runnable throws an uncaught exception.

Any task may create one or more tasks.

```
class Task {
    ...
    public abstract Task newTask(String name);
}
```

A task may listen to the outcome of another task, setting a listener. When a listener is set on a task, the task setting it will be used to execute the outcome notification: failed or completed.

```
class Task {  
    ...  
    public interface Listener {  
        public void failed(Task t, Throwable th);  
        public void completed(Task t, Object o);  
    }  
    public abstract void set(Listener l);  
}
```

Queue Brokers

Queue brokers are created with a unique name and owned by their creator tasks:

```
class Task {  
    ...  
    public abstract QueueBroker newBroker(String name);  
}  
  
interface QueueBroker {  
    String getName();  
    Task getTask();  
}
```

The rôle of queue brokers is to broker message queues between tasks, each queue being established between two tasks. The process is entirely asynchronous, based on matching connect requests with bind requests.

Any task may bind a port, on a broker, and any task may later unbind that port:

```
interface QueueBroker {  
    ...  
    public abstract boolean bind(int port, BindListener listener);  
  
    public abstract boolean unbind(int port);  
}
```

You will have noticed the bind listener given for bound port:

```
interface QueueBroker {  
    ...  
    public interface BindListener {  
        void accepted(MessageQueue queue);  
        void unbound();  
    }  
}
```

Important: the bind listener is always invoked on the task of the broker, that is, the task that created the broker, the once returned by `Broker.getTask()` Task.

The bind listener is notified of accepted connections and is given the corresponding message queues.

A new connection is requested as follows:

```
interface QueueBroker {  
    ...  
    boolean connect(String name, int port, ConnectListener listener);  
}
```

The given name is the name of the remote broker where the given port must be bound. If that remote broker cannot be found, the method `connect` returns `false`, otherwise it returns `true`.

You will have noticed the given listener:

```
interface QueueBroker {  
    ...  
    public interface ConnectListener {  
        void connected(MessageQueue queue);  
        void refused();  
    }  
}
```

Important: the bind listener is always invoked on the task of the broker, that is, the task that created the broker, the once returned by `Broker.getTask()` Task.

Message Queues

Queues are full-duplex point-to-point message-oriented queues. Hence, a queue supports both sending and receiving messages. Each queue is owned by the broker that created it.

```
public interface MessageQueue {  
    ...  
    abstract public QueueBroker broker();  
}
```

Any task may send a message through any message queue:

```
public interface MessageQueue {  
    boolean send(byte[] bytes, int offset, int length, SendListener l);  
}
```

A message is a range of contiguous bytes in an array. That range may not wrapped around:

- the offset must be 0 or greater.
- the (offset+length) must be smaller or equal to bytes.length

You will have noticed the given listener that returns the ownership of the sent range of bytes:

```
public interface MessageQueue {  
    public interface SendListener {  
        void sent(byte[] bytes, int offset, int length);  
    }  
}
```

Important: a sent range of bytes is owned by the queue until it has been returned, it may not be modified while the queue owns it.

The ownership of a sent range of bytes may be returned sooner or later that the corresponding receive on the other side of the queue. To receive messages, one has to set a listener, like this:

```
public interface MessageQueue {  
    ...  
    public interface Listener {  
        void received(byte[] msg);  
        void closed();  
    }  
    public abstract void setListener(Listener l);  
}
```

Nota Bene: the listener will be invoked on the task that was used to set the listener.

You will have noticed that the same listener is also used to notify that a message queue has been closed.

```
public interface MessageQueue {  
    ...  
    public abstract void close();  
    public abstract boolean closed();  
}
```

A queue may be closed at any time, by any task.

When closing a queue, invoking the method `close`, we need to distinguish between the two end points of that queue. There is the closed end point, the one on which the method `close` was invoked. There is also the closing end point, the one end point reacting to the closure request.

- The closed end point will no longer deliver messages or will drop any message sent, returning the ownership of the byte ranges of dropped messages.
- The closing end point will still deliver messages, ensuring that all messages sent before the close operation can be received. However, any message sent through the closing end point will be dropped, returning the ownership of the byte ranges of dropped messages.

Once the closing end point has received all messages sent on the closed end point, before it was closed, the closing end point becomes also closed.

It is accepted to close a queue multiple times.

It is accepted to close a queue that is the closing end point.